

An Input-Aware Sparse Tensor Compiler Empowered by Vectorized Acceleration

Xianhao He*, Haotian Wang^{*✉}, Jiapeng Zhang*, Wangdong Yang*,
Anthony Theodore Chronopoulos^{†‡} and Kenli Li^{*✉}

^{*}College of Computer Science and Electronic Engineering, Hunan University

[†] Department of Computer Science, The University of Texas at San Antonio

[‡] Department Computer Engineering and Informatics, University of Patras

{hexianhao, wanghaotian, zhangjp, yangwangdong}@hnu.edu.cn, antony.tc@gmail.com, lkl@hnu.edu.cn

Abstract—Sparsity is widely prevalent in real-world applications, yet existing compiler optimizations and code generation techniques for sparse computations remain underdeveloped. Sparse matrix-matrix multiplication (SpMM) is a representative operator in sparse computations, whose performance is often limited by the design of sparse formats and the extent of hardware architecture optimization. Most existing solutions achieve high-performance SpMM through two approaches: (1) meticulously designed kernels and specialized sparse formats, which require extensive manual effort, or (2) tensor compilers that support code generation, though these typically offer limited support for sparse patterns, making it challenging to adapt to complex sparsity patterns in practical applications. This paper presents SpMMTC, an input-aware sparse tensor compiler. Given a sparse matrix as input, SpMMTC analyzes its non-zero distribution and generates a vectorized kernel optimized for SpMM on the specific matrix. We evaluated SpMMTC on various workloads. It achieves speedups of 1.21x to 2.97x over state-of-the-art methods such as TACO, TVM, and ASPT on different multi-core processors. It also provides a speedup of up to 1.52x for sparse MobileNetV1 inference on the edge device.

I. INTRODUCTION

Sparsity is prevalent in deep learning applications, such as image classification [1], natural language processing [2], and object detection [3]. As DNN models grow, pruning weights becomes essential to reduce storage and computation, especially on resource-constrained devices [4]. Unstructured sparsity, a common pruning method, removes weights with minimal impact on accuracy, resulting in matrices with many zeros. Likewise, graph neural networks [5] often feature sparse adjacency matrices. Sparse matrix-matrix multiplication (SpMM) is a fundamental operator in sparse computation, multiplying a sparse matrix A with a dense matrix X to produce a dense matrix Y . SpMM is widely applied in areas like PageRank [6], recommender systems [7], and graph clustering [8].

The performance of SpMM is primarily influenced by two factors: 1) *sparse pattern* and 2) *hardware architecture*. The sparse pattern impacts memory access during computation, as the indices pointing to the dense matrix are non-contiguous, leading to a low data reuse ratio. The hardware architecture affects SpMM performance by enabling the development of high-performance parallel execution strategies that fully leverage the hardware's capabilities. Existing solutions for high-performance SpMM predominantly rely on runtime libraries.

These libraries require a carefully designed SpMM kernel and typically reorders the sparse matrices to improve data reuse before the kernel is executed. However, this often introduces cumbersome reordering algorithms and similarity measurements between rows or columns of the matrices [9]. Additionally, previous works, such as TC-GNN [10], aim to exploit dense blocks in sparse matrices to better utilize the tensor core units in NVIDIA GPUs, but this approach inevitably leads to unnecessary computations on zero elements, known as *redundant FLOPs* [11]. To achieve a high-performance SpMM kernel without extensive manual effort, tensor algebra compilers, such as TACO [12] and TVM [13], [14], have been proposed to automatically generate SpMM kernels. However, these compilers rarely analyze the sparse pattern of matrices, and TVM-style compilers often rely on long-term auto-tuning processes.

Therefore, to take both the sparse pattern and hardware architecture into consideration, we propose SpMMTC, an input-aware sparse tensor compiler for SpMM on multi-core CPUs. It takes the sparse matrix A and dimensions of dense matrix X as input, analyzes the non-zero distribution of A , then determines the optimal tile sizes along the direction of the rows of A and the columns of X based on the analysis results and backend architecture. Since the number of non-zeros in each row of A varies, the tiles of A may have unequal dimensions, which differs from the tiling techniques used in GEMM optimization [15]. SpMMTC generates high-performance kernels for these tiles that leverage SIMD units without redundant FLOPs. To accommodate the vectorized kernels, SpMMTC also generates transformation code to reorganize the data layout of the tiles. The generated code is assembled into several functions that are compiled into a library and invoked at runtime as needed.

In summary, our contributions in this paper can be concluded as the following:

- We propose SpMMTC, a sparse tensor compiler designed to automatically generate high-performance SpMM kernels tailored to specific sparse patterns, dimensions, and hardware architectures.
- SpMMTC analyzes the non-zero distribution of sparse matrices, optimizes memory access, and generates high-performance SpMM kernels with vectorized execution.
- We conducted experiments to evaluate SpMMTC on SpMM across various workloads on multi-core proces-

[✉]Corresponding Author

sors. The experimental results show SpMMTC achieves speedups of 1.21x to 2.97x over state-of-the-art baselines such as TACO, TVM, and ASpT on Intel and ARM multi-core processors. SpMMTC also provides a speedup of up to 1.52x for sparse MobileNetV1 inference on the widely-used edge device.

II. BACKGROUND AND MOTIVATION

SpMM is formulated as $Y = AX$, where A is sparse matrix with dimension $M \times K$, X is a dense matrix with dimension $K \times N$, and their calculation results are stored in dense matrix Y with dimension $M \times N$. The sparse A is usually stored in compressed sparse format like CSR and CSC. Algorithm 1 illustrates SpMM computation using the CSR format. A key challenge in SpMM is the indirect memory access in the dense matrix X (Line 4). Because the column indices of matrix A are non-contiguous, multiple rows of X are skipped as the iteration over the K -dimension progresses, leading to poor cache locality.

Algorithm 1 SpMM (Sparse Matrix Matrix Multiplication)

Input: CSR $A[M][K]$, float $X[K][N]$
Output: float $Y[M][N]$

```

1: for  $i = 0$  to  $A.num\_rows - 1$  do
2:   for  $k = A.ptr[i]$  to  $A.ptr[i + 1] - 1$  do
3:     for  $j = 0$  to  $N - 1$  do
4:        $Y[i][j] += A.values[k] * X[A.col\_idx[k]][j]$ 

```

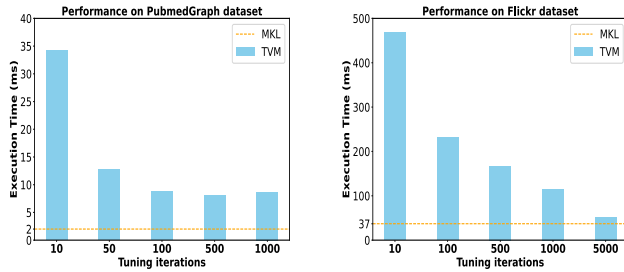


Fig. 1: Performance comparison between MKL and TVM for single-precision SpMM on x86 architecture

Recently, tensor algebra compilers such as TACO and TVM have been introduced to automatically generate high-performance kernels. These compilers generally adhere to the philosophy of separating the computation from the scheduling [16]. They provide a Domain-Specific Language (DSL) [17]–[19] that allows developers to independently define both the computation and the scheduling primitives of operations such as parallelizing, reordering, and unrolling for-loops. Moreover, TVM provides the auto-tuning ability to generate optimized, high-performance kernels. We compare the MKL [20] with TVM, using different tuning iterations, on the PubmedGraph and Flickr datasets¹ for SpMM on x86 architecture. The results are shown in Figure 1. As the number of iterations increases, the performance of TVM improves, but it remains lower

¹available at <https://networkrepository.com>

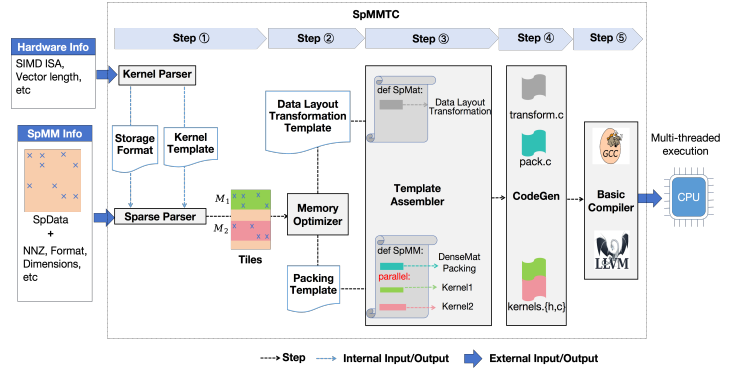


Fig. 2: Overview of SpMMTC. SpMMTC takes SpMM information, including the sparse matrix, storage format, dimensions of the two involved matrices, and other details as input, then optimizes and generates code tailored for the specific SpMM.

than MKL; additionally, the tuning process is time-consuming (e.g., tuning 5,000 iterations on the Flickr dataset takes over 10 hours). While compiler-based methods free developers from dealing with intricate hardware architecture and complex kernel designs, they struggle to analyze sparse patterns, which hinders further optimization of SpMM kernels.

Motivated from the above analysis, we design SpMMTC, an input-aware sparse tensor compiler to generate SpMM kernels and its overview is shown in Figure 2. SpMMTC follows five steps to generate SpMM kernels: 1) It takes SpMM and hardware information as input, analyzes the sparse patterns, and determines the storage format, kernel template, and sparse tiles for the computation; 2) It optimizes memory access for sparse and dense matrices and outlines the corresponding templates; 3) It assembles these separate templates into functions; 4) It generates C/C++ source code to instantiate the templates and implement these functions; and 5) All the code is compiled and executed on multi-core CPUs.

III. SPMMTC DESIGN

In this section, we detail SpMMTC, which is composed of the Kernel Parser, Sparse Parser, Memory Optimizer, Template Assembler, and CodeGen modules.

A. Kernel Parser

Kernel Parser determines how computations are performed on SIMD units and sketches out the sparse kernel template. Inspired by BLIS [15], the template uses an outer-product approach as shown in Figure 3. The advantages of outer product include a high computation-to-memory ratio (CMR) [21] and the elimination of redundant FLOPs.

1) *Storage Format*: The template utilized by Kernel Parser operates on tiles in A and X . To maintain the outer-product computation, it is essential to organize the sparse tiles of A in column-major order using three arrays: **tiles**, **indices**, and **data**, which are consistent with CSR or CSC formats. Each tile occupies a few entries in **tiles** that **tiles**[i][0] represent the offset in **indices** and **data**, **tiles**[i][1] indicates the starting row of the i -th tile. The indices of non-zero elements of this

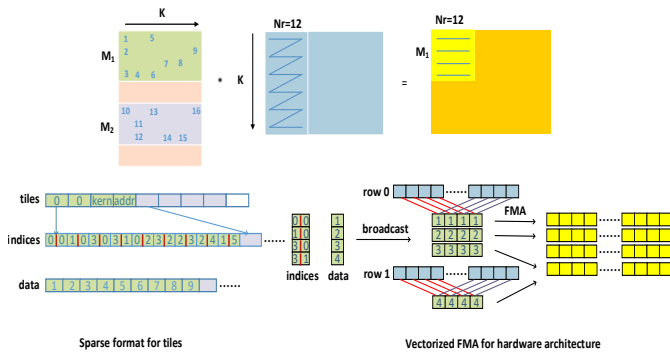


Fig. 3: Sparse kernel template. The numbers in the matrix represent non-zero elements. The black grids represent vector registers and we assume each vector register holds 4 elements. The sparse matrix is organized by three arrays: **tiles**, **indices** and **data**.

tile store from **indices**[**tiles**[i][0]] to **indices**[**tiles**[$i + 1$][0]], and **tiles**[i][2 :] stores the address of the kernel for this tile. Slightly different from CSR, each entry in **indices** is divided into two parts bitwise. The higher part stores the row index of the non-zero element within the tile, while the lower part stores its column index, facilitating the retrieval of the vector register used to store computation results. In our experiments, the type of **indices** is **int32_t** for single precision data and **int64_t** for double precision data, with the upper 5 bits used for the row index and the remaining lower bits for the column index. **data** stores the value of each non-zero element in the tile and corresponds to the entries in **indices**.

```

1 // template parameters
2 const int Mi = 4, N = 64, Nr = 12, d = 4, si = 2;
3 int j;
4 for (j = 0; j <= (N - Nr); j += Nr) {
5     int32x4_t loc;
6     float32x4_t sp, bd0, bd1;
7     // Following registers are determined by Mi and Nr
8     float32x4x3_t ds0, ds1, res0, res1, res2, res3;
9     int col_prev = -1;
10    for (int t = tiles[i][0]; t < tiles[i + 1][0]; t += d) {
11        sp = load(&data[t]), loc = vload(&indices[t]);
12        for (int m = 0; m < d; m += si) {
13            int idx0 = loc[m] >> 27;
14            int c0 = loc[m] & 0x7fffff;
15            bd0 = broadcast(sp[m]);
16            if (c0 > col_prev) ds0 = vload(&Xpack[c0][j:j+Nr]);
17            switch (idx0) case 0: res0 = vFMA(res0, bd0, ds0);
18            // omit other cases: 1, 2, 3
19            // next FMA
20            int idx1 = loc[m + 1] >> 27;
21            int c1 = loc[m + 1] & 0x7fffff;
22            bd1 = broadcast(sp[m + 1]);
23            if (c1 > c0) ds1 = vload(&Xpack[c1][j:j+Nr]);
24            else ds1 = ds0;
25            switch (idx1) case 0: res0 = vFMA(res0, bd1, ds1);
26            // omit other cases: 1, 2, 3
27            ds0 = ds1, col_prev = c1; // next iteration
28        }
29        // write back to output
30        int r = tiles[i][1];
31        vstore(&Y[r + 0][j], res0); vstore(&Y[r + 1][j], res1);
32        vstore(&Y[r + 2][j], res2); vstore(&Y[r + 3][j], res3); }
33    // handling the boundary of remaining 4 = (N - j)
34    ...

```

Listing 1: An instantiation of kernel template on **FP32** precision with an unrolled vectorized FMA

2) *Kernel Template*: Kernel Parser can determine the SIMD ISA and vector register length of the backend CPU architecture in advance. To implement high-performance sparse kernel, Kernel Parser outlines kernel template that utilizes SIMD instructions. For convenience, we assume d represents the number of elements each vector register can hold, the size of the i -th tile in A is (M_i, K) , and the tile size in X is (K, N_r) . Since the non-zeros of each tile are stored contiguously, every d non-zeros can be loaded into a vector register with a single load instruction. The computation involves multiplication of two tiles, requiring one vector register to store the broadcast values and N_r/d vector registers to store a row of the dense tile, with results maintained in $M_i \times N_r/d$ vector registers. N_r/d vectorized FMA operations are then performed on these three types of vector registers. Moreover, the corresponding indices are loaded into a vector register to determine which row of the dense tile to load and where to store the results in the vector registers. To maximize instruction-level parallelism, the template unrolls the for-loop associated with FMA operations by a factor of s_i , as shown in Listing 1.

3) *Optimization Goal*: The sparse kernel for the i -th tile is derived from kernel template by CodeGen module and is determined by the parameters M_i, N_r, d and s_i . Taking Listing 1 as an example, we assume the iteration over non-zeros of the i -th tile goes through T times (Line 10), the CMR can be calculated as follows:

$$\text{CMR} = \frac{\sum_{t=1}^T N_r \cdot d}{\sum_{t=1}^T [1 + 1 + N_r \cdot c(t)/d]} \quad (1)$$

$$= \frac{d^2}{2d/N_r + \sum_{t=1}^T c(t)/T}$$

where $c(t)$ represents at the t -th iteration, the number of distinct column indices in vector register loaded from **indices** array (Line 11), which determines the frequency of memory accesses to the rows of the dense tile (Lines 16 and 23). Additionally, since the number of vector registers is limited in each core, the vector registers used in our sparse kernel must satisfy the following constraints:

$$\begin{cases} 1 + 1 + s_i + \frac{N_r}{d} s_i + \frac{M_i N_r}{d} \leq \text{\#vec_reg} \\ s_i \geq 1, d \% s_i == 0, N_r \% d == 0 \end{cases} \quad (2)$$

where “1+1” indicates vector registers to store elements in **indices** and **data**, and the number of vector registers to store broadcasting values and dense tiles is multiplied by s_i due to the unrolling of for-loop (Lines 6 and 8). The optimization goal of sparse kernel is to maximize CMR while also maximizing s_i , subject to the constraints in (2). From equation (1), larger N_r and smaller $\sum_{t=1}^T c(t)/T$ contribute to greater CMR. In our experiments, we set $N_r = 16$ and 12 on x86 and ARM processors, respectively. The value of $\sum_{t=1}^T c(t)/T$ is related to the sparse tile size M_i and will be discussed later. After determining M_i and N_r , the maximum s_i is calculated as follows:

$$\begin{cases} s_i \leq (\text{\#vec_reg} - 2 - \frac{M_i N_r}{d}) / (1 + \frac{N_r}{d}) \\ s_i \geq 1, d \% s_i == 0 \end{cases} \quad (3)$$

Algorithm 2 Tile selection

Input: CSR $A[M][K]$
Output: #tiles N_t and row ranges $\{(S_1, E_1), \dots, (S_{N_t}, E_{N_t})\}$

```

1:  $nnz[i] \leftarrow \mathbf{ptr}[i+1] - \mathbf{ptr}[i]$  for  $i = 0, 1, \dots, M-1$ 
2:  $N_t \leftarrow 0, R \leftarrow \emptyset$ 
3:  $\mathbf{vis}[M] \leftarrow \{0, \dots, 0\}$   $\triangleright$  We assume  $\mathbf{vis}[-1] = \mathbf{vis}[M] = 1$ 
4: repeat
5:    $N_t \leftarrow N_t + 1$ 
6:    $r \leftarrow \arg \max_i (nnz[i] \ \& \ \mathbf{vis}[i] == 0)$ 
7:    $\mathbf{vis}[r] = 1$ 
8:    $S_{N_t} \leftarrow r, E_{N_t} \leftarrow r + 1$ 
9:    $\mathbf{best\_val} \leftarrow \sum_{t=1}^T c(t)/T$  based on the sparse tile with
     shape  $(E_{N_t} - S_{N_t}, K)$  and row range  $(S_{N_t}, E_{N_t})$ 
10:  repeat
11:     $S = (\mathbf{vis}[S_{N_t} - 1] == 0) ? S_{N_t} - 1 : S_{N_t}$ 
12:     $E = (\mathbf{vis}[E_{N_t}] == 0) ? E_{N_t} + 1 : E_{N_t}$ 
13:     $s_1 \leftarrow \sum_{t=1}^T c(t)/T$  based on the sparse tile with
       shape  $(E_{N_t} - S, K)$  and row range  $(S, E_{N_t})$ 
14:     $s_2 \leftarrow \sum_{t=1}^T c(t)/T$  based on the sparse tile with
       shape  $(E - S_{N_t}, K)$  and row range  $(S_{N_t}, E)$ 
15:    if  $s_1 \geq \max\{s_2, \mathbf{best\_val}\}$  then
16:       $\mathbf{best\_val} \leftarrow s_1, S_{N_t} \leftarrow S, \mathbf{vis}[S_{N_t}] = 1$ 
17:    else if  $s_2 \geq \max\{s_1, \mathbf{best\_val}\}$  then
18:       $\mathbf{best\_val} \leftarrow s_2, E_{N_t} \leftarrow E, \mathbf{vis}[E_{N_t} - 1] = 1$ 
19:    else if  $\mathbf{best\_val} > \max\{s_1, s_2\}$  then
20:      break
21:  until  $M_{N_t} \leftarrow E_{N_t} - S_{N_t} + 1$  violates the constraint
     (2) and (3)
22:   $R.append(S_{N_t}, E_{N_t})$ 
23: until  $\forall i \in [0, M-1], \mathbf{vis}[i] = 1$ 
24: return  $N_t, R$ 
```

tile, with shape (M_i, K) , minimizes $\sum_{t=1}^T c(t)/T$ in equation (1) to achieve optimized CMR.

To address this, Sparse Parser first calculates the number of non-zeros in each row (e.g., $\mathbf{ptr}[i+1] - \mathbf{ptr}[i]$ in CSR format), then adopts a greedy strategy to choose tiles, as shown in Algorithm 2. The algorithm takes sparse storage format like CSR as input and returns the number of tiles N_t along with their row ranges $\{(S_1, E_1), \dots, (S_{N_t}, E_{N_t})\}$, where (S_i, E_i) indicate the range of the i -th tile. Intuitively, a tile composed of rows with more non-zero elements is likely to reduce the number of distinct columns in the vector register and increase the number of for-loop iterations, resulting in a smaller $\sum_{t=1}^T c(t)/T$. Thus, the algorithm begins with the available row that has the maximum number of non-zeros when identifying a new tile (Lines 6~8). The algorithm tentatively expands the current tile by adding its last row or next row, then simulates the procedure of sparse kernel on the expanded tile to calculate $\sum_{t=1}^T c(t)/T$. The algorithm records the best result and expands the tile accordingly. If the calculations from the tentatively expanded tile are less than the recorded best result, or if expanding by one row would violate the constraint in (2), the shape and row range of the current tile are determined, and the algorithm proceeds to find the next tile.

C. Memory Optimizer

1) *Data Layout Transformation:* Conventionally, sparse matrices are stored in CSR or CSC formats. To adapt the storage format to our sparse kernel, Memory Optimizer outlines a template for data layout transformation, which is later instantiated by the CodeGen module. Listing 2 illustrates the instantiation of transformation template from CSR format shown in Figure 3. Variables such as **NNZ**, **Nt**, **t_info** are template parameters populated at CodeGen stage. **t_info[i][0]** and **t_info[i][1]** store the row range of the i -th sparse tile, while **t_info[i][2:]** stores address of the specific sparse kernel to be invoked during the computation of the sparse tile. Memory optimizer employs min heap to ensure the non-zeros in the tile are organized in column-major reorder.

2) *Dense Matrix Packing:* The above descriptions focus on the optimization of the sparse matrix, but optimizing the data layout of the dense matrix is also important. Memory Optimizer also outlines a template for packing dense matrix into tiles with shape (K, N_r) . The benefit of packing is to improve the locality of dense tiles in sparse kernel. The template packs dense matrix as shown in Figure 4. Specifically, each thread is responsible for packing one row. Since both the dense matrix and tiles are typically stored in row-major order, this row packing effectively leverages cache locality.

D. Template Assembler

So far, there are three types of templates proposed by SpMMTC. Since these templates are separate, Template Assembler assembles them into two types of functions: sparse matrix definition (**SpMat**) and **SpMM** kernel, as shown in Figure 2. The specific code for these functions is generated by the CodeGen module. The **SpMat** function internally utilizes the

```

1  const int NNZ = 16, Nt = 2;
2  int t_info[Nt][4] = {{0, 4, 0xb23e0854, 0xaaaa}, {6, 9, 0
   xb23e086c, 0xaaaa}};
3  int tiles[Nt][4], indices[NNZ], off = 0;
4  float data[NNZ];
5  for (int i = 0; i < NTile; i++) {
6    tiles[i][0] = off; tiles[i][1] = t_info[i][0];
7    tiles[i][2] = t_info[i][2]; tiles[i][3] = t_info[i][3];
8    // first: idx of csr_indices, second: row idx
9    // comparing order: csr_indices[first], row idx
10   MinHeap<int, int> heap;
11   for (int j = t_info[i][0]; j < t_info[i][1]; j++) {
12     heap.push({csr_ptr[j], j});
13   }
14   while (!heap.empty()) {
15     int idx, row = heap.pop();
16     indices[off] = csr_indices[idx];
17     data[off++] = csr_data[idx];
18     if (idx + 1 < csr_ptr[row + 1]) {
19       heap.push({idx + 1, row});
20     }
21   }
22 }
```

Listing 2: An instantiation of data layout transformation template

B. Sparse Parser

Since kernel template is based on tiles from sparse matrix, Sparse Parser aims to select the sparse tiles so that the i -th

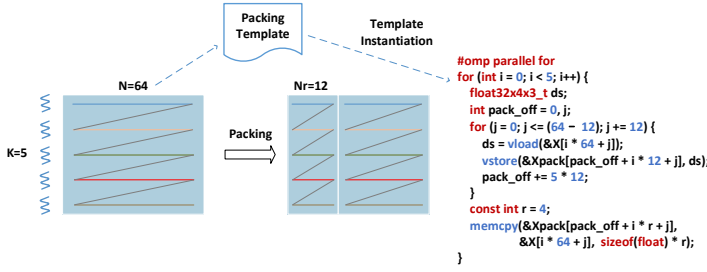


Fig. 4: Packing of a dense matrix and instantiation of a template with dimensions 5×64 and **FP32** precision. Each thread is responsible for packing one row.

data layout transformation code to convert the original sparse format (e.g., CSR, CSC) into the structures **tiles**, **indices**, and **data**, as previously described, and returns a handler specifying these structures. Users must call this function before executing the **SpMM**. When invoking the **SpMM** function, users provide the handler and the dense matrix as inputs. The function first packs the dense matrix and then calls the appropriate sparse kernel based on the current tile being processed.

E. Code Generation

The CodeGen module generates code to implement the **SpMat** and **SpMM** functions defined by the Template Assembler. For dense matrix packing, the code can be directly generated based on the dimensions of the dense matrix and the packing template. For sparse matrices, the sizes of each tile are determined using Algorithm 2, where the size of the i -th tile with a row range (S_i, E_i) is calculated as $M_i = E_i - S_i$. CodeGen then generates transformation code using the data layout transformation template proposed by the Memory Optimizer, as shown in Listing 2. Furthermore, based on the constraint in (3), SpMMTC selects the maximum unroll factor s_i to generate code with kernel template. The generated code consists of C/C++ source files, and the corresponding functions are compiled into a dynamic library, enabling users to invoke them at runtime.

IV. EXPERIMENT

A. Setup

We evaluate SpMMTC on x86 and ARM multi-core architectures: Intel Xeon Gold 5120, Kunpeng 920 (KP920) and Raspberry Pi 4 (RPI 4). In our experiments, the two servers are equipped with 56 and 64 cores, respectively, each with 256 GB of RAM, while the embedded device has 4 cores and 4 GB of RAM. We implement SpMMTC in C/C++, and the generated code is compiled using GCC with the $-O3$ optimization flag.

B. Benchmarks

We conduct SpMM evaluations on two workloads: (I) sparse matrices in scientific computation and (II) sparse matrices in deep learning. For SpMM in scientific computation, we select datasets as shown in Table I, and for SpMM in deep learning, we use sparse matrices of ResNet50 model with magnitude pruning and random pruning [22] provided by deep learning

TABLE I: Sparse matrices for SpMM evaluation. These datasets are available at <https://sparse.tamu.edu/> and <https://networkrepository.com>

Name	rows	nnz
Flickr	89,250	899,756
com-Youtube	1,134,890	5,975,248
GAP-road	23,947,347	57,708,624
Reddit	232,965	114,615,892
Bump_2911	2,911,419	127,729,899
vas_stokes_4M	4,382,246	131,577,616
nlpkt160	8,345,600	225,422,112
Queen_4147	4,147,110	316,548,962
mycielskian19	393,215	903,194,710
uk-2005	39,459,925	936,364,282

matrix collection (DLMC) [23]. The following state-of-the-art libraries or compilers are used as baselines: 1) MKL, a linear algebra library optimized for x86 architectures by Intel; 2) ASPT [9], a high-performance sparse computation library that enhances data reuse by reordering rows; 3) TACO, a tensor compiler for linear algebra; 4) TVM, a tensor compiler with auto-tuning capabilities; and 5) XNNPack [24], a deep learning library providing SpMM interface.

C. Performance Analysis

1) *Workload I*: We evaluate these methods on SpMM using both FP32 and FP64 precisions, setting the columns of dense matrices (N) to 512. The experiments are performed in a multi-thread environment (56 threads on Intel and 64 threads on KP920) and their results are shown in Figure 5. ASPT demonstrates superiority over other baselines by reordering rows of sparse matrices before execution, thereby improving data reuse. Among the two tensor compilers, TVM outperforms TACO in most datasets due to its auto-tuning capability for generated kernels. On the Intel server, SpMMTC achieves up to 419.52 GFLOPs (FP32) and 200.78 GFLOPs (FP64), surpassing the baselines with an average speedup of 2.46x over MKL, 1.30x over ASPT, 3.11x over TACO, and 2.97x over TVM. On the ARM server, where XNNPack is used instead of MKL, SpMMTC achieves up to 299.65 GFLOPs (FP32) and 177.79 GFLOPs (FP64), outperforming all baselines with an average speedup of 1.99x over XNNPack, 1.48x over ASPT, 2.08x over TACO, and 1.95x over TVM, similar to its performance on the x86 architecture. Additionally, SpMMTC selects each optimal tile in at most 28 iterations on x86 architectures and 8 iterations on ARM architectures (Lines 10~21 in Algorithm 2), making it more efficient than other tensor compilers' auto-tuning process.

2) *Workload II*: Unlike scientific computation, matrix dimensions in deep learning are smaller. We also evaluate SpMMTC performance under this scenario on Intel and KP920 servers using FP32 precision. Figure 6 shows the evaluation results on the sparse matrices of ResNet50. The model is pruned under varying sparsity levels with magnitude pruning and random pruning, comprising a total of 540 matrices.

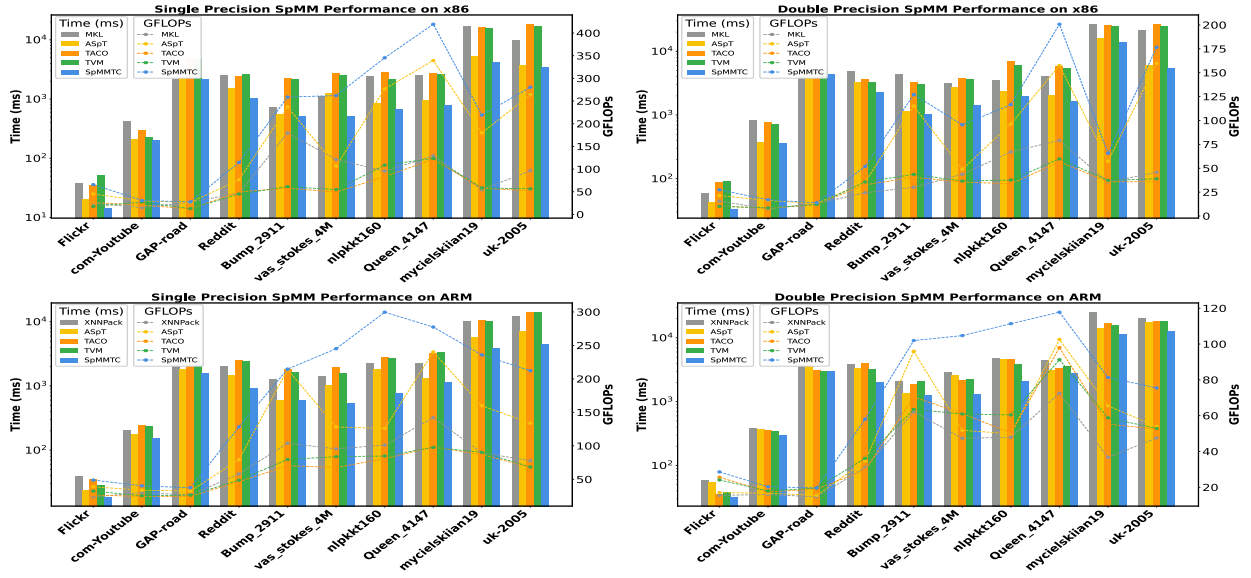


Fig. 5: SpMM performance of *Workload I* on x86 (Top) and ARM architectures (Bottom).

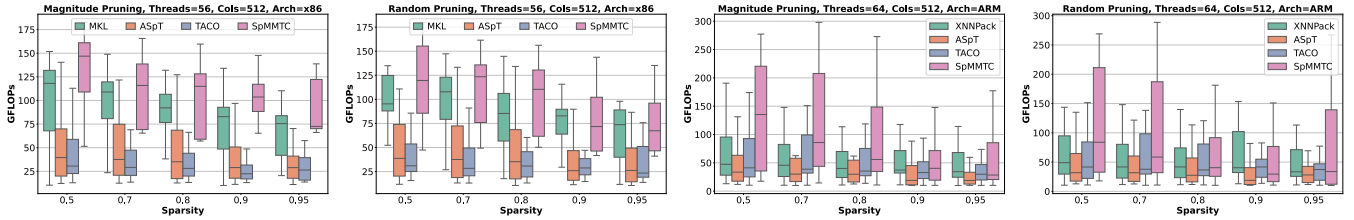


Fig. 6: SpMM performance of pruned ResNet50 on x86 and ARM architectures.

These matrices come from sparse filters, where convolution operations can be converted into matrix multiplications with im2col operation. The evaluations are conducted in a multi-thread environment, with the columns of the dense matrix set to 512 as well. The experimental results show that the throughput of these methods is affected by sparsity. As sparsity increases, their throughput decreases accordingly. In *Workload I*, ASPT demonstrates advantages over other baselines, but it is less efficient for deep learning computation workloads. SpMMTC outperforms all baselines on both architectures, achieving an average speedup of 1.21x over MKL, 1.84x over ASPT, 1.39x over XNNPack, and 2.28x over TACO.

MobileNetV1, we replace sparse convolution operations with im2col operations using our SpMM and evaluate end-to-end inference under different sparsity levels on the RPi 4 device. We compare SpMMTC with XNNPack, and the results are shown in Figure 7. SpMMTC achieves a speedup of up to 1.52x over XNNPack under sparsity levels ranging from 0.7 to 0.9.

V. CONCLUSION

In this paper, we propose SpMMTC, a sparse tensor compiler designed to automatically optimize and generate SpMM code. SpMMTC takes a sparse matrix and the dimensions involved in SpMM as inputs, analyzes the sparse patterns of the matrix, and generates high-performance kernels specialized for SpMM with identical dimensions and sparse matrix, leveraging vectorized acceleration on multi-core architectures. In our experiments, SpMMTC outperforms state-of-the-art methods, including MKL, ASPT, TACO, and TVM.

ACKNOWLEDGEMENT

This work was partially supported by National Key R&D Program of China (Grant No. 2023YFB3001805), CCF-Huawei Populus Grove Fund (CCF-HuaweiSY202407), Programs of NSFC (Grant No. U21A20461, U24A20235, 62321003, 62227808), China Postdoctoral Science Foundation (2024M750879), China National Postdoctoral Program for Innovative Talents (BX20240111), and Changsha Natural Science Foundation (kq2502277).

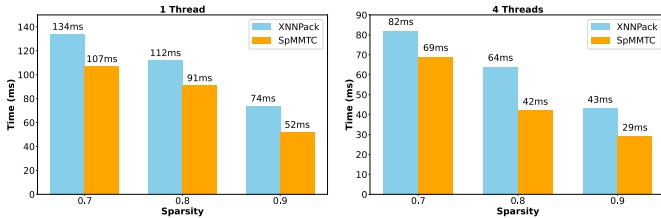


Fig. 7: End-to-end inference time of pruned MobileNetV1 on RPi 4 device.

3) *MobileNetV1 on edge*: MobileNetV1 is a well-known DL model widely used in image classification and object detection on mobile or edge devices. Optimizing the model's performance can help improve user experience and battery energy consumption. To evaluate SpMMTC performance on

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Red Hook, NY, USA, 2017, p. 6000–6010.
- [3] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *IEEE transactions on pattern analysis and machine intelligence*, vol. 39, no. 6, pp. 1137–1149, 2016.
- [4] W. Niu, X. Ma, S. Lin, S. Wang, X. Qian, X. Lin, Y. Wang, and B. Ren, "Patdnn: Achieving real-time dnn execution on mobile devices with pattern-based weight pruning," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 907–922.
- [5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [6] M. Yang, H. Wang, Z. Wei, S. Wang, and J.-R. Wen, "Efficient algorithms for personalized pagerank computation: A survey," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [7] H. Chen, Y. Bei, Q. Shen, Y. Xu, S. Zhou, W. Huang, F. Huang, S. Wang, and X. Huang, "Macro graph neural networks for online billion-scale recommender systems," in *Proceedings of the ACM on Web Conference 2024*, 2024, pp. 3598–3608.
- [8] A. Hajiveisheh, S. A. Seyed, and F. A. Tab, "Deep asymmetric nonnegative matrix factorization for graph clustering," *Pattern Recognition*, vol. 148, p. 110179, 2024.
- [9] C. Hong, A. Sukumaran-Rajam, I. Nisa, K. Singh, and P. Sadayappan, "Adaptive sparse tiling for sparse matrix multiplication," in *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, 2019, pp. 300–314.
- [10] Y. Wang, B. Feng, Z. Wang, G. Huang, and Y. Ding, "{TC-GNN}: Bridging sparse {GNN} computation and dense tensor cores on {GPUs}," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 149–164.
- [11] L. Wilkinson, K. Cheshmi, and M. M. Dehnavi, "Register tiling for unstructured sparsity in neural network inference," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1995–2020, 2023.
- [12] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, "The tensor algebra compiler," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–29, 2017.
- [13] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "{TVM}: An automated {End-to-End} optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594.
- [14] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen *et al.*, "Ansor: Generating {High-Performance} tensor programs for deep learning," in *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, 2020, pp. 863–879.
- [15] F. G. Van Zee and R. A. Van De Geijn, "Blis: A framework for rapidly instantiating blas functionality," *ACM Transactions on Mathematical Software (TOMS)*, vol. 41, no. 3, pp. 1–33, 2015.
- [16] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," *Acm Sigplan Notices*, vol. 48, no. 6, pp. 519–530, 2013.
- [17] P. Liu, A. J. Root, A. Xu, Y. Li, F. Kjolstad, and A. J. Bik, "Compiler support for sparse tensor convolutions," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, 2024. [Online]. Available: <https://doi.org/10.1145/3689721>
- [18] A. Dias, L. Anderson, K. Sundararajah, A. Pelenitsyn, and M. Kulkarni, "Sparseauto: An auto-scheduler for sparse tensor computations using recursive loop nest restructuring," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Oct. 2024. [Online]. Available: <https://doi.org/10.1145/3689730>
- [19] J. Liu, Z. Zhao, Z. Ding, B. Brock, H. Rong, and Z. Zhang, "Unisparse: An intermediate language for general sparse format customization," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA1, Apr. 2024. [Online]. Available: <https://doi.org/10.1145/3649816>
- [20] E. Wang, Q. Zhang, B. Shen, G. Zhang, X. Lu, Q. Wu, and Y. Wang, "Intel math kernel library," *High-Performance Computing on the Intel® Xeon Phi™: How to Fully Exploit MIC Architectures*, pp. 167–188, 2014.
- [21] W. Yang, J. Fang, D. Dong, X. Su, and Z. Wang, "Libshalom: Optimizing small and irregular-shaped matrix multiplications on armv8 multi-cores," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–14.
- [22] M. Zhu, T. Zhang, Z. Gu, and Y. Xie, "Sparse tensor core: Algorithm and hardware co-design for vector-wise sparse neural networks on modern gpus," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 359–371.
- [23] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse gpu kernels for deep learning," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–14.
- [24] Google, "Xnnpack," <https://github.com/google/XNNPACK>.